Ex 6	Semantic Movie Recommendation System using Vector Database

Learning Objective

Implement a scalable semantic search engine that utilizes high-dimensional vector embeddings and a vector database (**FAISS**) to retrieve contextually relevant media from a large-scale dataset.

Question

Consider you are a Lead Data Engineer at CineMatch, a rapidly growing streaming platform. The current search system uses **Keyword Matching**, which means if a user searches for *"movies about space exploration,"* they only get results if the exact words like *"space"* or *"exploration"* appear in the title.

The Marketing Team reports that users are frustrated because they cannot search based on **themes, emotions, or concepts**. For example:

- Searching *"heartwarming stories"* does not return movies like *Toy Story*
- Searching *"AI and robots"* may miss relevant movies without those exact keywords

Your objective is to design a **Semantic Search Prototype** using the MovieLens dataset that understands **meaning instead of exact words**.

Your Solution Must

- Convert movie titles and genres into **high-dimensional vector embeddings** so that semantically similar words (e.g., *Galaxy* and *Space*) are close in vector space
- Use **FAISS** for efficient similarity search across large datasets
- Ensure fast retrieval even when the dataset scales to thousands or millions of movies
- Combine multiple features (Title + Genres) into a single representation for better context understanding
- Use cosine similarity (or equivalent) to measure closeness between movies and user queries

Tasks to Perform

1. Install required libraries and set up the environment for embedding generation and similarity search
2. Download the MovieLens dataset and load the movie data into a structured format
3. Clean and preprocess the dataset by handling missing values and formatting text fields

4. Combine relevant features such as **Title and Genres** into a single searchable text representation
5. Generate high-dimensional embeddings for each movie using a pre-trained embedding model
6. Store all embeddings in a **FAISS** index for efficient similarity search
7. Maintain metadata (movie ID, title, genres) linked to each embedding
8. Accept a natural language query from the user (e.g., *"a movie about friendship and adventure"*)
9. Convert the user query into an embedding using the same model
10. Perform similarity search in FAISS to retrieve the top relevant movies
11. Extract and display the matched movie titles along with similarity scores
12. Evaluate the system by comparing keyword-based results vs semantic search results

Code:

```
import numpy as np
import pandas as pd
import faiss
import requests, zipfile, io, warnings
from sentence_transformers import SentenceTransformer
warnings.filterwarnings("ignore")
print("Downloading MovieLens dataset...")
url  = "https://files.grouplens.org/datasets/movielens/ml-latest-small.zip"
r    = requests.get(url)
z    = zipfile.ZipFile(io.BytesIO(r.content))
z.extractall(".")
print("✓ Dataset downloaded\n")
df = pd.read_csv("ml-latest-small/movies.csv")
df.dropna(inplace=True)
df["genres"] = df["genres"].str.replace("|", " ", regex=False)
df["title"]  = df["title"].str.strip()
print(f"✓ Loaded {len(df)} movies")
print(df.head(3))
df["text"] = df["title"] + " " + df["genres"]
print("\nSample combined text:")
print(df["text"].head(3).to_string())
print("\nLoading embedding model...")
model      = SentenceTransformer("all-MiniLM-L6-v2")   # lightweight, fast
print("✓ Model loaded. Generating embeddings...")
embeddings = model.encode(df["text"].tolist(), show_progress_bar=True)
```

```
embeddings = np.array(embeddings).astype("float32")
# normalize for cosine similarity
faiss.normalize_L2(embeddings)
print(f"✓ Embeddings shape: {embeddings.shape}")
dim   = embeddings.shape[1]
index = faiss.IndexFlatIP(dim)       # Inner Product = cosine after normalization
index.add(embeddings)
print(f"✓ FAISS index built with {index.ntotal} vectors\n")
metadata = df[["movieId", "title", "genres"]].reset_index(drop=True)
def semantic_search(query, top_k=5):
    q_vec = model.encode([query]).astype("float32")
    faiss.normalize_L2(q_vec)
    scores, indices = index.search(q_vec, top_k)
    results = metadata.iloc[indices[0]].copy()
    results["similarity"] = scores[0]
    return results
def keyword_search(query, top_k=5):
    words   = query.lower().split()
    mask    = df["text"].str.lower().apply(lambda t: any(w in t for w in words))
    results = df[mask][["title", "genres"]].head(top_k).copy()
    results["similarity"] = "N/A"
    return results
queries = list(input("Enter queries: ").split(","))
for query in queries:
    print("\n" + "═" * 60)
    print(f"  QUERY: \"{query}\"")
    print("═" * 60)
    print("\n🔵 KEYWORD SEARCH:")
    kw = keyword_search(query)
    if kw.empty:
        print("  No results found.")
    else:
        for _, row in kw.iterrows():
            print(f"  • {row['title']:<45} | {row['genres']}")
    print("\n🟢 SEMANTIC SEARCH:")
    sem = semantic_search(query)
    for _, row in sem.iterrows():
        print(f"  • {row['title']:<45} | Score: {row['similarity']:.4f} | {row['genres']}")
```

Output:

```
Enter queries: AI and robots, movies about space exploration

═══════════════════════════════════════════════════════════
  QUERY: "AI and robots"
═══════════════════════════════════════════════════════════

🔵 KEYWORD SEARCH:
  • Waiting to Exhale (1995)                    | Comedy Drama Romance
  • Tom and Huck (1995)                         | Adventure Children
  • Dracula: Dead and Loving It (1995)          | Comedy Horror
  • Cutthroat Island (1995)                     | Action Adventure Romance
  • Sense and Sensibility (1995)                | Drama Romance

🟢 SEMANTIC SEARCH:
  • Robot Overlords (2014)                      | Score: 0.5188 | Action Adventure Sci-Fi
  • Robot & Frank (2012)                        | Score: 0.4984 | Comedy Drama Sci-Fi
  • A.I. Artificial Intelligence (2001)         | Score: 0.4895 | Adventure Drama Sci-Fi
  • I, Robot (2004)                             | Score: 0.4648 | Action Adventure Sci-Fi Thriller
  • Robot Jox (1990)                            | Score: 0.4424 | Sci-Fi

═══════════════════════════════════════════════════════════
  QUERY: " movies about space exploration"
═══════════════════════════════════════════════════════════

🔵 KEYWORD SEARCH:
  • Lawnmower Man 2: Beyond Cyberspace (1996)   | Action Sci-Fi Thriller
  • Something to Talk About (1995)              | Comedy Drama Romance
  • Much Ado About Nothing (1993)               | Comedy Romance
  • Thirty-Two Short Films About Glenn Gould (1993) | Drama Musical
  • Space Jam (1996)                            | Adventure Animation Children Comedy Fantasy Sci-Fi

🟢 SEMANTIC SEARCH:
  • Lost in Space (1998)                        | Score: 0.6388 | Action Adventure Sci-Fi
  • SpaceCamp (1986)                            | Score: 0.6312 | Adventure Sci-Fi
  • Plan 9 from Outer Space (1959)              | Score: 0.6223 | Horror Sci-Fi
  • The Space Between Us (2016)                 | Score: 0.6136 | Adventure Sci-Fi
  • Space Cowboys (2000)                        | Score: 0.6124 | Action Adventure Comedy Sci-Fi

═══════════════════════════════════════════════════════════
CONCLUSION:
  Keyword search misses conceptually related movies.
  Semantic search retrieves movies by meaning & context.
═══════════════════════════════════════════════════════════
```

Learning Outcome

Upon completion of this program, students will be able to:

- Understand the limitations of traditional keyword-based search systems
- Transform unstructured text into **high-dimensional vector embeddings**
- Use **FAISS** to perform fast similarity-based retrieval
- Design a **semantic search pipeline** that captures meaning and context
- Apply vector search techniques to real-world applications like recommendation systems and search engines